THOMAS MORE

# Internship Assignment Tokheim

Realization Document

Sybren Backx
Student Bachelor Applied Computer Science– Application Development

2025 – 2026

# Inhoudsopgave

# 1. Introduction

This document describes the realization of the internship project, covering the tasks and milestones completed during the internship period. Following the initial project plan, which outlined the scope, this document provides a technical explanation of how that plan was put into practice. To maintain a clear narrative, topics are presented chronologically across three main parts: context about the company and their tools, a feasibility study on the assignment and the development of the dashboard map. By the end of this document, you will have a detailed overview of the technical challenges encountered and the solutions implemented during the time spent at Tokheim. This document is intended for the jury that will be evaluating the internship at the end of the period

## 1.1. Abbreviations

| | |
|---|---|
| **OP** | ONE Portal, management platform for customers to manage their stations. |
| **PM** | Petrol Manager, old management platform which serves API endpoints. |
| **SC** | Station Configurator, module to configure, add and update stations. |
| **CouCom** | Country Company a network or group of stations. |
| **DFS** | Dover Fueling Solutions. |
| **Geocoding** | Translating addresses to coordinates or the reverse called reverse geocoding. |
| **Nominatim** | An open-source geocoding tool using OpenStreetMap (OSM) data. |
| **OPT** | Outdoor Payment Terminal. |

## 1.2. Tokheim/DFS Solutions

The internship took place at Tokheim, a worldwide manufacturer, active in the petroleum industry. They provide a variety of products, ranging from fueling dispensing equipment to payment terminal software. They have a long-standing history in the market with the founder John J. Tokheim. He started the company in 1901 and patented the world's first vehicle fuel dispenser. Tokheim specializes in providing "Station-in-a-Box" solutions, allowing retailers to source nearly all their equipment from a single provider.

*1 Tokheim logo*

Since 2016, Tokheim operates as a core brand within **Dover Fueling Solutions (DFS)**, a division of the Dover Corporation, side by side to other major brands like Wayne Fueling Systems

Some of their products:

- **Fuel Dispensers**: The flagship **Quantium series** multiple fuel types.
- **Retail Automation**: Systems like the **Prizma POS** and **DFS Anthem UX** platform manage sales, inventory, and customer engagement.
- **Payment Solutions**: Secure terminals such as **Crypto VGA** and **Tokheim OASE** for multiple payment methods, including contactless transactions.
- **Clean Energy**: Expansion into alternative fuels including **AdBlue**, **CNG**, **LNG**, **Hydrogen**, and **EV charging** infrastructure.

*2 Quantium series fuel dispenser*

**Market Position & Future**
As part of DFS, which reported approximately **$7.7 billion in revenue** for 2024, Tokheim is a leading player in a market valued at approximately $2.82 billion globally. The company is currently pivoting toward digital transformation and sustainability, investing in **IoT-enabled remote monitoring** and infrastructure for the energy transition away from fossil fuels. (Matrix, 2025)

## ONE PORTAL

Clients currently use ONE Portal, an Angular-based management tool, to oversee operations via several modules:

- Stock module to manage their stock supply
- Pricing module to set the prices of the products in store
- Sales module to get statistics and insights into their sales

Most of my internship focused on the alerts module. This module provides clients with detailed real time messages and warnings about the status of their stations. For instance, when a station's OPT (Outdoor Payment Terminal) is out of service. Then a red alert warning will show up in the One Portal alerts module. Or if a fuel tank is almost empty, an automatic event is sent and automatically acted upon, to schedule a delivery.

Right now, the alerts module is displayed in a tree view, where you can drill down on CouComs to get the station alert details of a station. Admin users see all stations displayed in the view, but customers only see a few CouComs that belong to them. This means the stations are prefiltered based on which user is logged in.

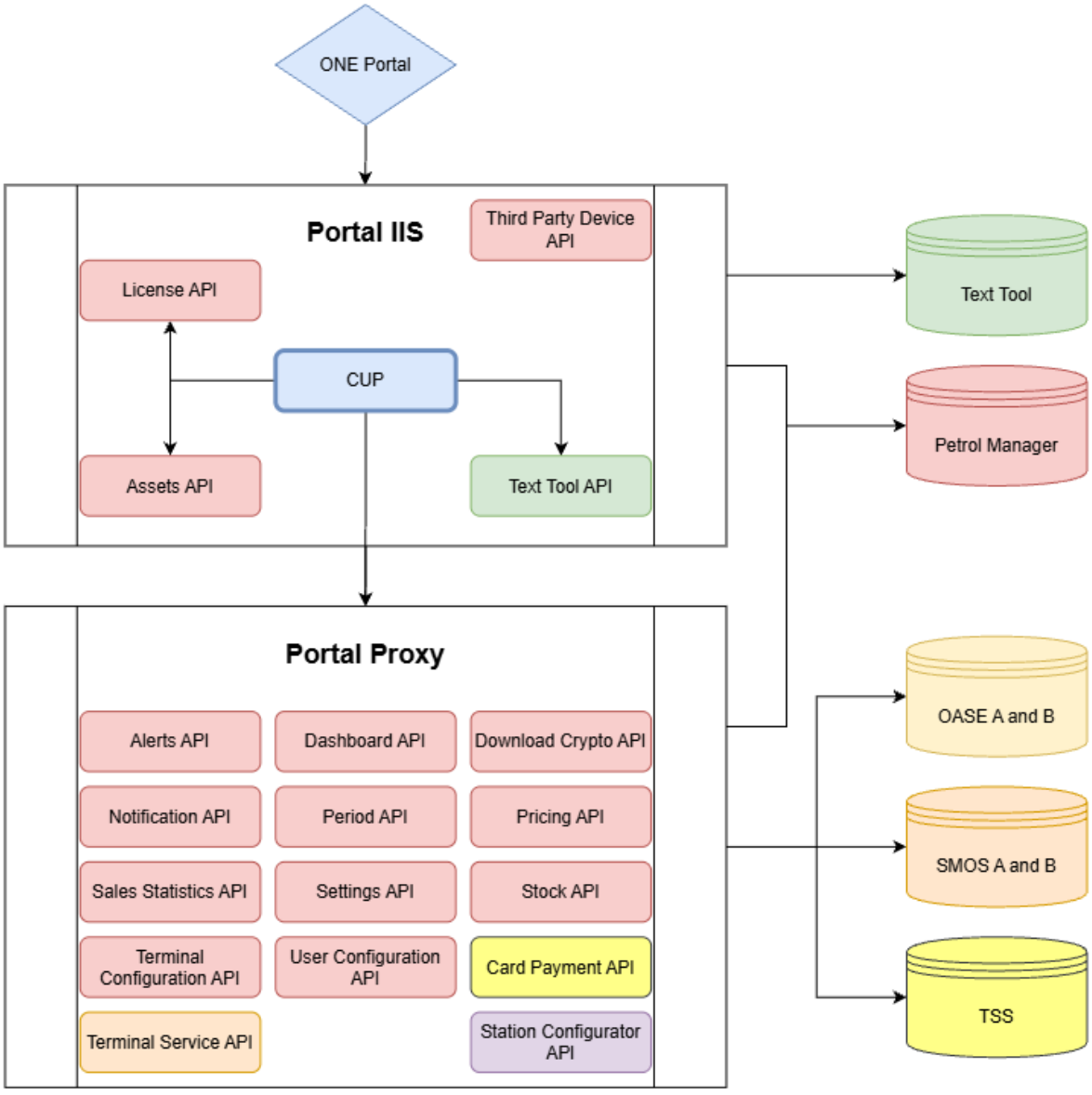
The main goal of the alerts module is to help clients keep an eye on their stations, help them spot issues fast and act quickly on them.

## PETROL MANAGER

Petrol Manager is the old tool that is still gradually being replaced by One Portal. Most current functionalities in ONE Portal originate from the PM environment.

## ONE PORTAL API AND DATABASE SCHEMA

This is an schematic overview of how all the databases and API's are connected in One Portal.



# 2. Feasibility Study on Geocoding Tools

To kick off the internship, it was essential to assess the project's feasibility. Since an interactive map requires station coordinates that were not initially available, an analysis was performed on the existing dataset.

The database of Petrol Manager already contained a lot of data, specifically station address data. Based on the address, zip code, city and country, a conversion tool can be made to translate this data into geographic coordinates. The practice of converting address data into longitude and latitude is called geocoding. And here the first challenge was found. The first challenge was the scale: approximately 32,000 stations needed to be geocoded to populate the interactive map.

This feasibility study evaluates several geocoding tools based on cost, coverage, and performance to determine the most viable solution.

## 2.1. Comparison of geocoding tools

| Tool | Coverage | Cost | Speed / Method |
|---|---|---|---|
| **HERE** | Worldwide | Paid (Pay-as-you-grow) | High-speed bulk geocoding |
| **US Census** | US Only | Free | Batch records (10k at a time) |
| **Nominatim** | Worldwide | Free (Open Source) | 1 req/sec (Public API) or Unlimited (Local) |
| **QGIS (MMQGIS)** | Worldwide | Free | Bulk geocoding via plugins |
| **Geoapify** | Worldwide | Paid (Credit-based) | Bulk geocoding API |

#### HERE

In this table we are looking for the most cost-effective tool that covers the whole world and preferably is also fast. The first tool, named “here”, is a company that provides a large scale of geocoding related services. But to use their geocoding services, they use a Pay-as-you-grow pricing model. So, to convert 32000 stations, this would become too pricey. (HERE, 2026)

#### US CENSUS

Census offers a free batch record converter service of 10,000 records at a time. The only downside is that they are only focused locations based in the US. So, this tool can only convert a small number of the stations. (Census, n.d.)

### NOMINATIM OPENSTREETMAP

Nominatim is an open-source tool utilizing OpenStreetMap data. For frequent bulk conversions, Nominatim recommends installing their database locally. But this installation requires a lot of disk space. For the full installation of the whole planet, you would need at least 1TB of hard disk space. Even on a well configured machine the import of a full planet takes around 2-3 days.

They do offer an interesting alternative. Namely, a public API endpoint (a web address that accepts requests and returns data), where a maximum of 1 request per second is allowed to translate a record. This could be used to set up a script and translate each station one by one. This will be more time costly than a simple bulk conversion, but the script only needs to finish once in our case. (nominatim, Open-source geocoding with OpenStreetMap data, n.d.)

### MAPSCAPING IN QGIS WITH MMQGIS PLUGIN

QGIS (formerly Quantum GIS, where GIS stands for Geographic Information System) is a free, open-source desktop Geographic Information System application used to create, edit, visualize, analyze, and publish geospatial information on Windows, macOS, and Linux. It has a rich library of plugins, like MMQGIS, to provide what you need. This software does require some more setup, but it does allow you to freely bulk convert records. (ODonohue, 2025 )

### GEOAPIFY

Similar to HERE, Geoapify offers bulk conversion services but operates on a credit-based pricing model, which was less favorable for this specific project compared to free open-source alternatives. (Geoapify, 2026)

## 2.2. Conclusion

The public API from Nominatim OpenStreetMap and the open-source QGIS software were chosen as the most viable tools for this use case. Due to being free, having worldwide coverage and they can quickly be set up locally.

It was decided to proceed with a Nominatim-based script. Although the initial processing takes several hours, it is a stable, cost-free solution. For future scalability, newly added stations can be geocoded individually via a Nominatim API call within the Station Configurator module in ONE Portal. This ensures that the map remains up to date as the network grows.

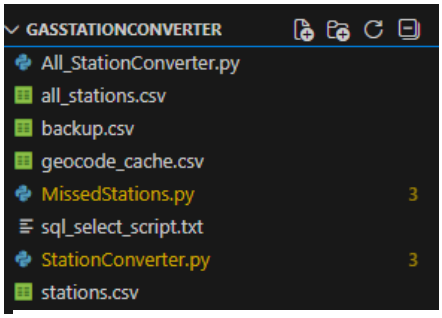
*Note Whilst evaluating tools, the QGIS website was temporarily unavailable, which drove me to the choice to focus on a Nominatim integration.*

## 2.3. Nominatim OpenStreetMap Script

To convert the station addresses into geographic coordinates, a custom geocoding script was developed using Python. This script sends a query containing station data to the Nominatim API endpoint and ideally it returns the coordinates of that station. Sometimes the coordinates can't be found, and a different search query is used.

### RATIONALE FOR PYTHON AND PANDAS

The decision to use Python was based on the efficiency and flexibility of the Pandas library (a popular library for exploring and manipulating big data). Since bulk conversion of 32,000 records was a one-time task, a standalone Python script allowed for rapid development and delivery. The tool was faster and easier to set up compared to configuring it inside One Portal or Petrol Manager. At the time, we also did not know if it was possible to perform the geocoding, so our main focus was more aimed at figuring out if it was possible. Python, in combination with pandas, served as a sufficient and efficient way for geocoding the stations.




*1 Station converter script*

### WORKFLOW

**Data Preparation:**
A dataset was exported from the PMConfiguration database using SQL Server Management Studio. This export combined the CouCom table (for country data) with the Station table (for name, address, zip code, and city), needed to get the parameters for the geocoding.

**Geocoding Process:**
The script loads the excel spreadsheet into a Pandas Data Frame and iterates through each row. Processing each row one by one. For every station, it constructs a query and sends it to the Nominatim API.

**Data Retrieval:**
The API returns a detailed JSON response, from which the script extracts the specific latitude and longitude.

**Output:**
The results are saved to a new file, stations_geocoded.csv, containing the original station data, alongside the retrieved coordinates.

**Handling Discrepancies:**
To manage stations that could not be automatically located, a second script called MissedStations.py was developed. This utility filters the results to identify records with missing coordinates. By isolating these "blind spots," we can manually verify the addresses or investigate why Nominatim failed to find a match. To analyze coordinates, cross-referencing was done on problematic addresses with the Nominatim web interface and Google Maps.

| B | C | D | E | F | G | H | I | J |
|---|---|---|---|---|---|---|---|---|
| stationId | coucom | station | address | zip | city | country | lat | lon |
| 11 | Tango Nederland | Tango Breda | De Struijckenstraat 246 | 4812BL | Breda - Tango | Netherlands | | |
| 28 | Tango Nederland | Tango Udenhout | Kreitmolenstraat 203 | 5071MD | UDENHOUT - TANGO | Netherlands | | |
| 29 | Tango Nederland | Tango Veendam Uoydsweg | Uoydsweg t.o. 3537 | 2641KJ | VEENDAM - TANGO | Netherlands | | |

*2 Excel file containing stations with missing coordinates*

Also, an important notice is that there are usage policies coupled to the use of Nominatim and OpenStreetMap:

Nominatim Search Tool Website (nominatim, nominatim search, n.d.)
OSM usage policy (osmfoundation, n.d.)

For example, you can only send one request per second. Otherwise, your address could get flagged and blocked.

For long-term integration, an alternative geocoding solution must be investigated. While the current Python script served the initial data migration, an automated solution integrated directly into OP and PM would be superior to a manual external tool for future station configurations.

## 2.4. Some issues with the conversion

The following is a summarization of issues found whilst converting the stations. In these scenarios, it leads to no coordinates being found and getting an empty string back. Or the wrong coordinates are sent back.

#### COUNTRY AND CITY ARE PREFIXED WITH COMPANY NAMES MOST OF THE TIME

615b Hatertseweg,6533GL, Nijmegen - Tango, Tango Nederland (Wrong)
615b Hatertseweg,6533GL, Nijmegen, Nederland (Correct)

#### THE ADDRESS IS SPELLED WRONG

60 3 Tango Uithoorn Amsterdamsweg 9 1422AC UITHOORN – TANGO
60 3 Tango Uithoorn Amsterdamseweg 9 1422AC UITHOORN

#### THE ADDRESS IS SPELLED WRONG

11 3 Tango Breda De Struijckenstraat 246 4812BL – Breda - Tango
11 3 Tango Breda Dr. Struyckenstraat 246 4812BL Breda

#### IN SOME CASES, 2 DIFFERENT STATIONS CAN BE FOUND

23,Kreitmolenstraat 203,5071MD,UDENHOUT ,,Tango Veendam Lloydsweg

#### UNRECOGNIZABLE CHARACTERS IN DIFFERENT LANGUAGES

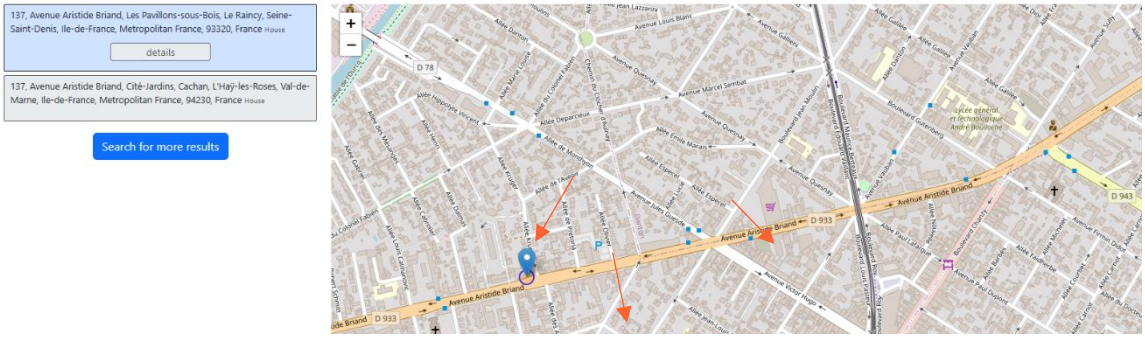
rue francois mitterand
rue françois MITTERAND

#### LAZY INPUTS

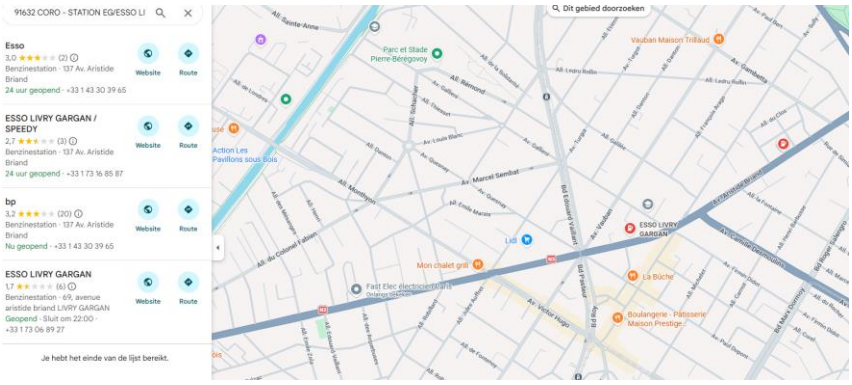
Lexo 0033 Mombasa,"Mombasa country, 12345

#### INACCURATE RESULTS BECAUSE OF WRONG HOUSE NUMBERS

STATION EG/ESSO LIVRY GARGAN,"137 AV ARISTIDE BRIAND, 93190",
48.9098730, 2.5067330




*3 Nominatim search result to the left of the crossroad*




*4 Google Maps search result showing stations on the right of the crossroads*

## SOLUTIONS

Data can be scrubbed to get more accurate results in Nominatim OpenStreetMap. Getting coordinates can be done based on different fields. Most of the locations can be found combining the address and zip code alone. With city and country there are mixed results as they need to be scrubbed first, to not include the company name. Sometimes, coordinates can be found by adding the station name alone where the address and zip fields fail.

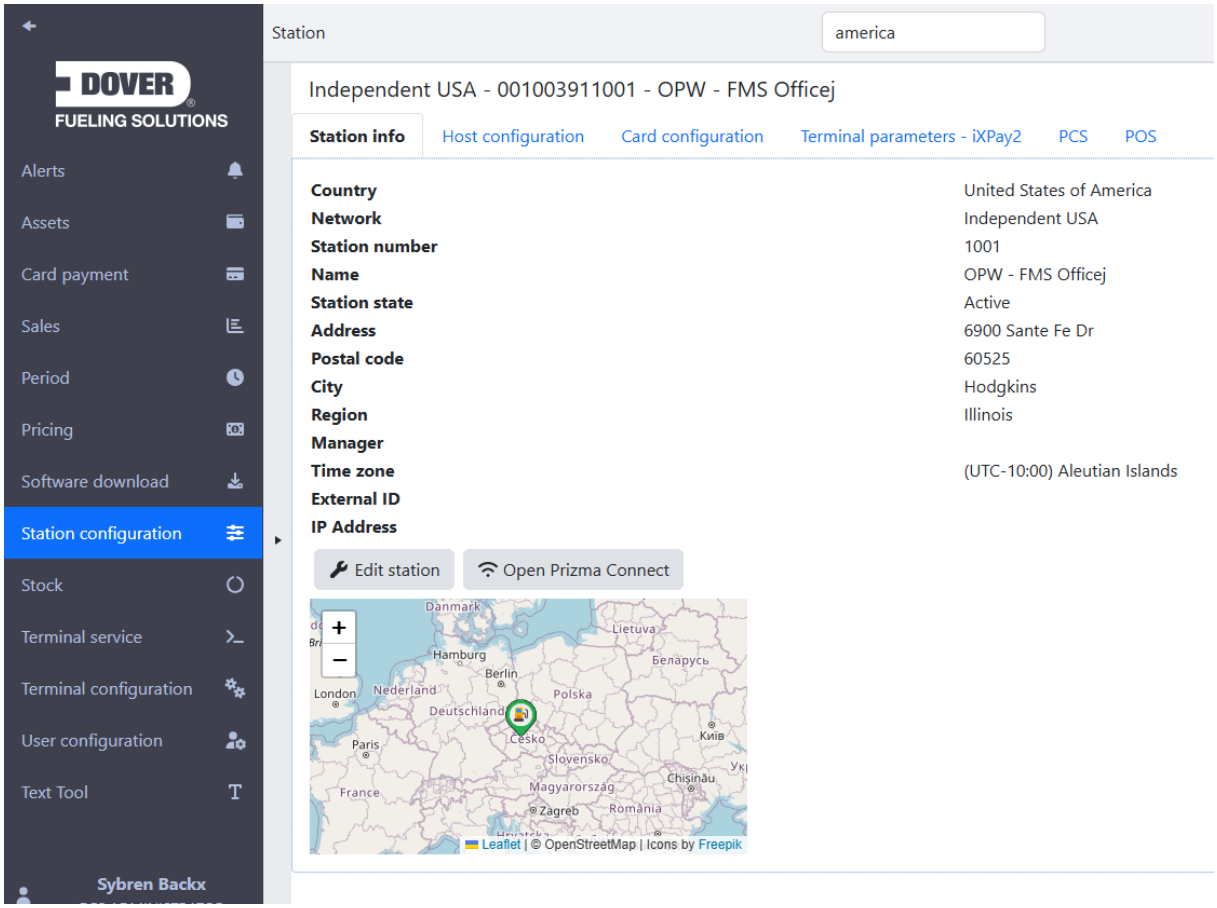
## QUERY PARAMETER STRINGS:

1. {address}, {zip} {city}, {country} (Full precision)
2. {address}, {zip} {city} (Regional focus)
3. {address}, {zip} (Postal code focus)
4. {station_name} (Brand search)

## 2.5. Conclusion

The feasibility study confirms that geocoding the station CouComs using Nominatim is a valid strategy. It will take a while to run the script, and the accuracy will heavily depend on the quality of the data and the search query string. But this can be partially solved by scrubbing the data and adjusting the script. Or by providing a solution after the conversion, so the coordinates can be updated

To address any remaining wrongly configured stations, an interactive map was integrated into the Station Configurator. By providing a map on the station details page, users receive immediate visual feedback on the marker's location. A drag-and-drop functionality will allow users to manually refine coordinates by moving the marker to the correct position, ensuring the right coordinates to be set.




*5 Leaflet map inside the SC module*

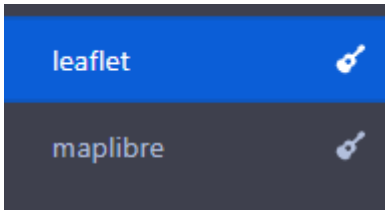
# 3. Comparison study on map libraries

Previously we concluded in the Feasibility Study of converter tools, that it is possible to continue with the assignment. In the following section, you will be met with a comparison study on different mapping libraries to display the interactive map in OP. After this study, all the tools and libraries are known, and a conclusion is formed about which library best suits our use case.

## 3.1. Overview

To perform a comparison study, map building libraries were researched. The most viable ones were tested in the OP dashboard to see any advantages and disadvantages.
To decide on the best library, the following criteria were held in mind:

- How fast is the map at rendering tiles.
- Is the map user friendly
- What range of functionalities/plugins does the library offer
- How easy it is to maintain the code of the map
- The size of the library




*6 Map libraries*

### LEAFLET

For the use case of developing a dashboard, Leaflet is more than powerful enough. The other libraries are faster, but for a simple dashboard with max 200 plotted station, Leaflet is more than sufficient. The other libraries like Maplibre GL JS, for example, are meant for more heavy-duty applications like Uber or Waze.

The library is smaller than the rest and the map itself, with all its features, is very easy to set up. Compared to other libraries, Leaflet is way less complex, and the learning curve is smaller.


*7 Leaflet prototype*

Another plus, Leaflet is self-contained. It does not rely on any third-party dependencies. This means the installation is a lot more straightforward. There are less security risks this way. Leaflet also won't break because of a mismatch in versions of dependencies. Taking everything into account, Leaflet is a very suitable candidate for this assignment. (Agafonkin, leafletjs, 2010–2025)

### MAPLIBRE GL

The Maplibre GL library provides a lot of freedom in the way you can customize your maps. It does have a steep learning curve both to set up a map and to add functionality to it. For this assignment, Maplibre GL is a bit overkill. It is more used for enterprise level applications where the core business revolves around mapping and tracking real time data.

Their documentation is well maintained, but unfortunately hard to access. They have a lot of examples available, but their documentation is kept in a bundle you must install and run on docker.

Based on this, the Maplibre Gl library can be used for our use case, but it would be excessive. A lot of functionalities would go unused.

### OPENLAYERS

The Openlayers library is similar to the Maplibre GL library, in that it mainly serves another purpose than what is needed for the assignment. Openlayers is used for complex GIS (Geographic Information System) operations to capture, manage, analyze, and map spatial data.

The assignment does not include complex data visualization. Therefore, openlayers does not fit our use case. It does provide faster rendering and more customization features, just like Maplibre GL. But most of these features will go unused.

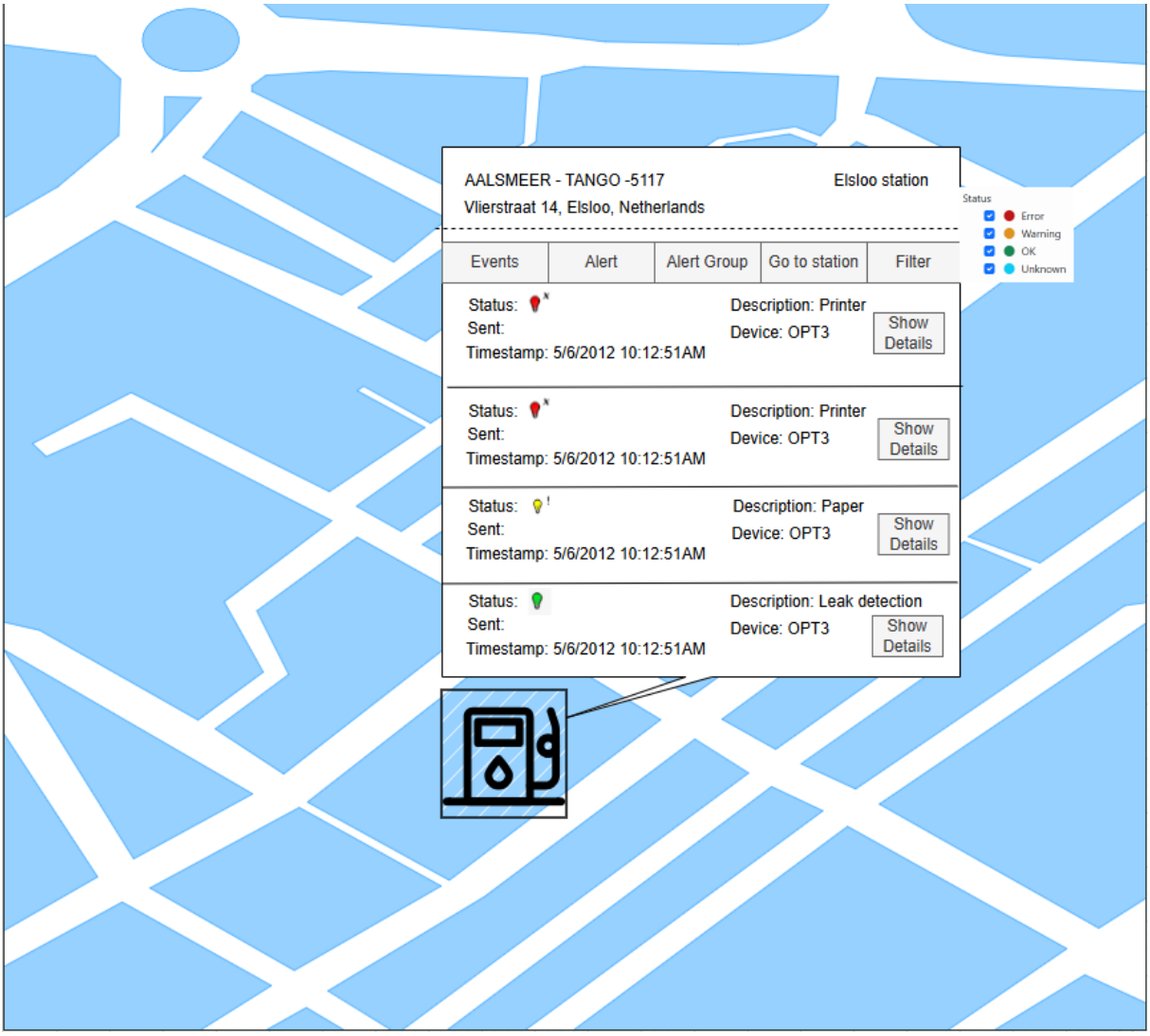
### SUMMARY

Leaflet is ideal for simple, lightweight interactive maps and quick integrations like this assignment.

Maplibre gl provides high-performance vector rendering and custom styling for modern, dynamic applications.

Openlayers is perfect for advanced GIS features, geospatial analysis, and enterprise-level mapping projects.
(Geoapify, Map libraries comparison: Leaflet vs MapLibre GL vs OpenLayers - trends and statistics, 2020)

## COLLECTING ALL FIELDS NEEDED TO DISPLAY ON THE MAP




*8 high level map marker popup screen prototype*

This serves as a high-level example of what information will be displayed when clicking a map marker. The code from the alerts module will be reused where possible, specifically it's API calls in the cup-alerts service file. This file should already contain everything that would need to be displayed on the map. The map should not replace the whole alerts module. The client should be easily redirected to the alerts module when he wants to see details of a specific alert on a station. Should this redirect prove to not be possible, then alternatively, more information will need to be displayed inside a map station marker.

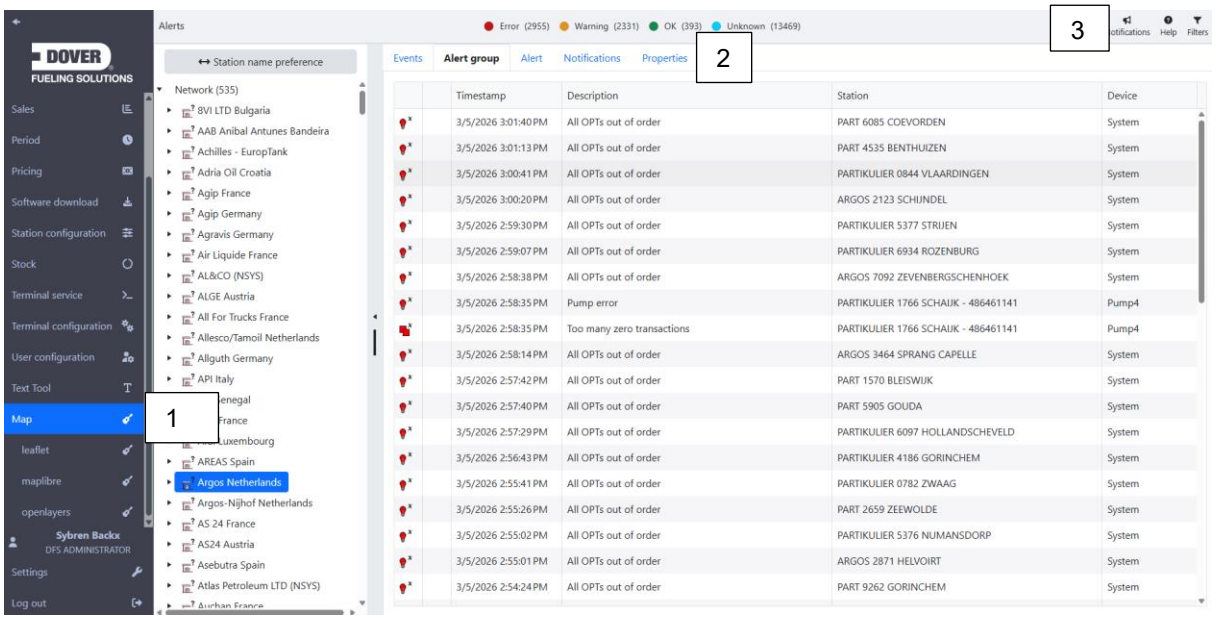
### FIELDS NEEDED TO DISPLAY ON THE MAP

| Field | Type | Table |
|---|---|---|
| **stationId** | Int | Station |
| **CouComId** | Int | Station |
| **CouComName** | String | CouCom |
| **StationNumber** | Int | Station |
| **StationName** | String | Station |
| **Address** | String | Station |
| **City** | String | Station |
| **GpsLatitude** | Int | Station |
| **GpsLongitude** | Int | Station |
| **AlertGroupId** | Int | AlertGroup |
| **AlertGroupLevel** | Int | AlertGroup |

## SETTING UP THE MAP IN ONE PORTAL

Setting up the map in One Portal will require some thought up front. There are multiple options on where the map can be displayed. The possible options are as follows:

- As part of the navigation menu [1]
- As part of the tab navigation bar inside the alerts module [2]
- As a button on the top right next to the filter option [3]




*9 UI map button options*

**Option 1: Primary navigation menu**
It looks the best, but there would have to be a differentiation in the future, should other modules also want to integrate with a map, for example the stock module.

**Option 2: Tab navigation within the alerts module**
This option does not fit in with the rest of the menu items and is not recommended because it would have to function in the tab view. So, it would not fit in the right place in the code too.

**Option 3: Action button (top-right) next to filters**
This option seems like the cleanest option. This way the button can be used in correspondence with the tree view, which would be ideal for using the existing filtering of CouComs.

### COUPLING THE ALERTS MODULE

Once the map is set up, part of the alerts module functionality will be reused inside the map. Namely, the map marker will be tied to the alert levels. For example, a station with an error will show up in red, a station with a warning will show up in yellow and stations that are running without any alerts are shown on the map in green.

When a station is clicked, most of the information displayed will be like what is shown in the alert module. The space to display information is limited though. So, the map will not become as descriptive as the alerts module. Probably the information on Network and Alert Group level will be shown. Not on individual station level. This all depends on whether it is possible to do a redirect from the station marker to the right station in the alerts module.

## 3.2. Conclusion

Based on this comparison study, Leaflet is the most suitable library. While MapLibre GL and OpenLayers are better for heavy-duty vector rendering and advanced GIS analysis, their complexity does not match the specific requirements of this assignment.
Leaflet excels in the criteria most relevant to our use case:

**Performance**:
With a maximum of 200 plotted stations, Leaflet's rendering speed is more than sufficient, making the high-performance engines of other libraries unnecessary.

**Maintainability**:
It is self-contained and has no third-party dependencies, so it minimizes security risks and versioning conflicts, ensuring long-term stability.

**Ease of Use:**
The low learning curve allows for faster development and easier handovers in the future.

**Decision:**
The project will proceed with **Option 3**, implementing the map as a toggleable view integrated with the existing tree-view filtering system.

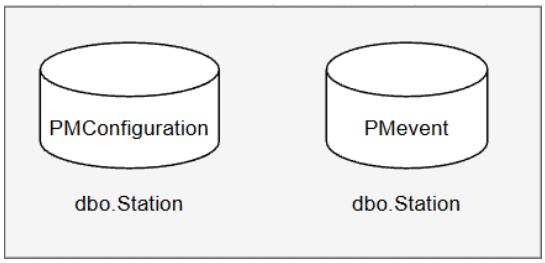
Leaflet is selected as the optimal mapping library for the One Portal (OP) dashboard, offering the best balance between simplicity, performance, and maintainability for displaying approximately 200 station markers. While MapLibre GL and OpenLayers provide high-end, complex GIS capabilities, they are over-engineered for this case compared to Leaflet's lightweight, self-contained architecture. Based on a technical comparison and "Proof of Concept" testing, the project will implement Leaflet integrated with the current alerts module.

# 4. Petrol Manager and Station Configurator

This section covers the modifications made in Petrol Manager (PM) and the Station Configurator (SC) to support the map functionality. To ensure the map receives accurate, real-time data, changes were implemented across the database layer, stored procedures, and API endpoints.

## Database Overview

PMConfiguration is a collection of many databases like the Station database, which in turn holds all the information on how a station is configured. The same holds true for PMEvent, which holds all the information on events happening on a station.




To manage all this data, stored procedure scripts are used. These scripts serve to set up and initialize the databases when the startup script is run. This startup script takes all stored procedures and executes them one by one, thus initializing and rebuilding the databases. This means that any changes need to be made inside the stored procedure scripts to make these changes permanent. If, for example, new tables or fields are added directly to the database, then upon restarting the database and rerunning the stored procedures, the changes will be lost.

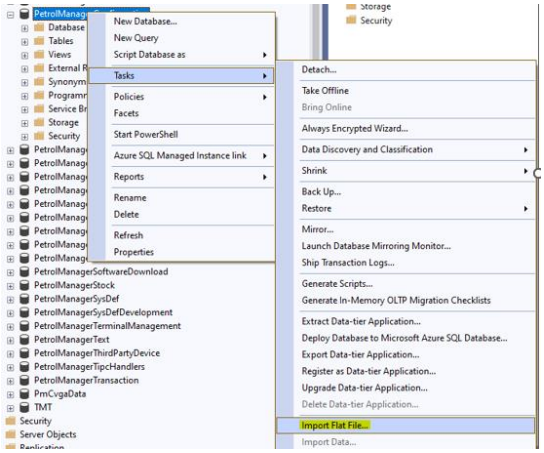
**Changes Overview:**

- **Database Schema:** Added GpsLatitude and GpsLongitude columns to the Station tables in both PMConfiguration and PMEvent.
- **Stored Procedures:** Updated selection and update scripts (e.g., Up_SelectStationForCouComWithFilter) to include the new coordinate and address fields.
- **API Extensions:** Modified existing endpoints to return coordinate data to the frontend and handle geocoding logic during station creation.
- **Logic Integration:** Embedded the Nominatim API within the SC workflow to automate coordinate retrieval.

The changes in PMEvent are to support the new GpsLongitude, GpsLatitude and Address fields. The API call that is done when selecting a CouCom or expanding a network in the tree is extended to return these additional fields.
The changes in PMConfiguration are to support the creation and storing of new stations and updating stations inside the OP Station Configurator module.

## 4.1. Storing translated coordinates

To move the geocoded stations with their coordinates to the PMConfiguration database. The best strategy a team member and I proposed would be to create a temporary SQL table to store the longitude and latitude in. After that it is possible to match both tables on stationId in a query to get the coordinates in the right place. This seems to be the safest option, because the database can always be rolled back to the previous state. My team member already added the necessary columns to store the coordinates in.




*10 Proposal to get geocoded stations back into the database, Wim van den Hurk*

## 4.2. New coordinate fields in PM

To build the map, stations will need to be requested from the PM API together with their coordinates. To support this, the PM database was extended with the fields GpsLatitude and GpsLongitude inside the station table of the PetrolManagerConfiguration and PetrolManagerEvent database.

| GpsLatitude | GpsLongitude |
|---|---|
| 51.813296 | 5.8355741 |
| 52.2865704 | 4.5929216 |
| 52.591859 | 6.2804259 |
| 50.9499412 | 5.7851233 |

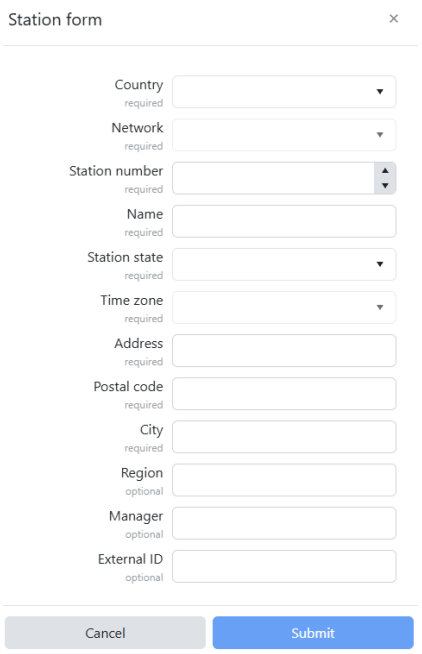
*11 Coordinates tables*

## 4.3. Station Configurator Extension

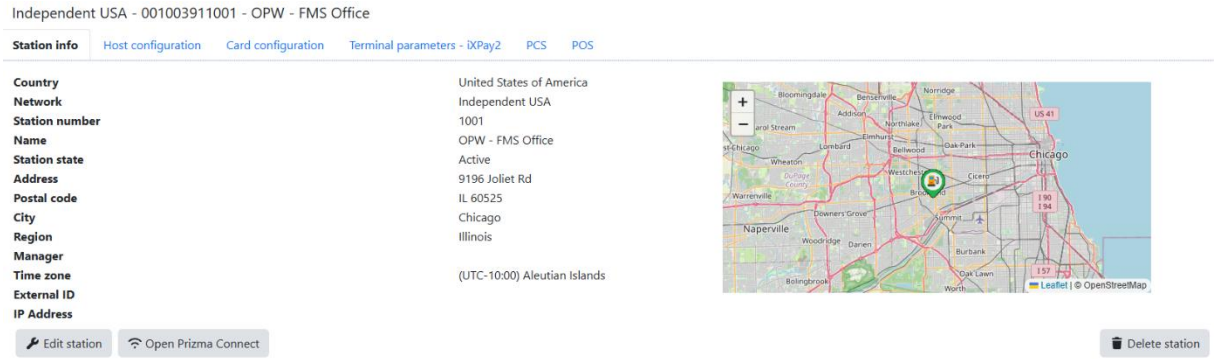
Inside the Station Configurator module in OP, it is possible to create and update stations. These functionalities were extended to automatically provide the stations with coordinates. By re-using the Nominatim API, the coordinates can be found. When a user fills in the form, when creating or updating a station, and submits the form, then automatically a call is done to the Nominatim API to get the coordinates.

The address, postal code and country name fields are used to build a query and send it to the Nominatim API. Then the coordinates that come back are added to the PM configuration database.

Upon creating a station, the API call is always done, and the coordinates are always updated. When updating a station, the API call is only done when the address, postal code or country has changed.

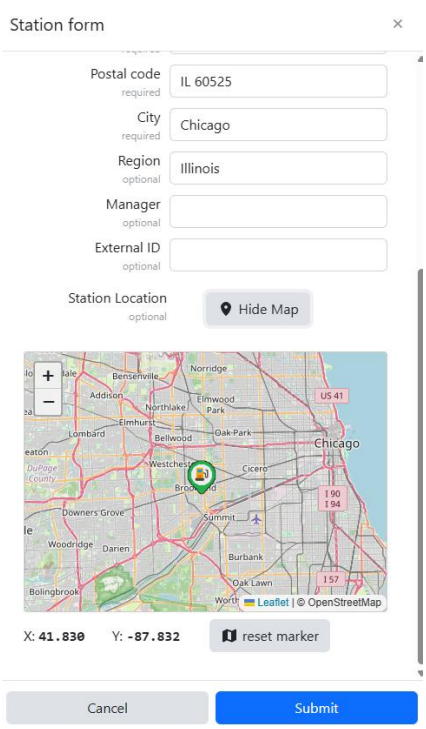



*12 SC station form*




*13 SC station info panel*

To correct any missing or incorrect coordinates from Nominatim, The SC was extended with a separate map. Now, inside the update form, exists a map where it is possible to drag and drop the station marker to its position. Then when submitting the form, the coordinates are updated based on where you put the marker. This way it is possible to correct any misconfigured stations. The map needs to be toggled in order to see it, and a reset button is provided, should the marker be moved accidentally.




*14 SC update form with marker dragging*

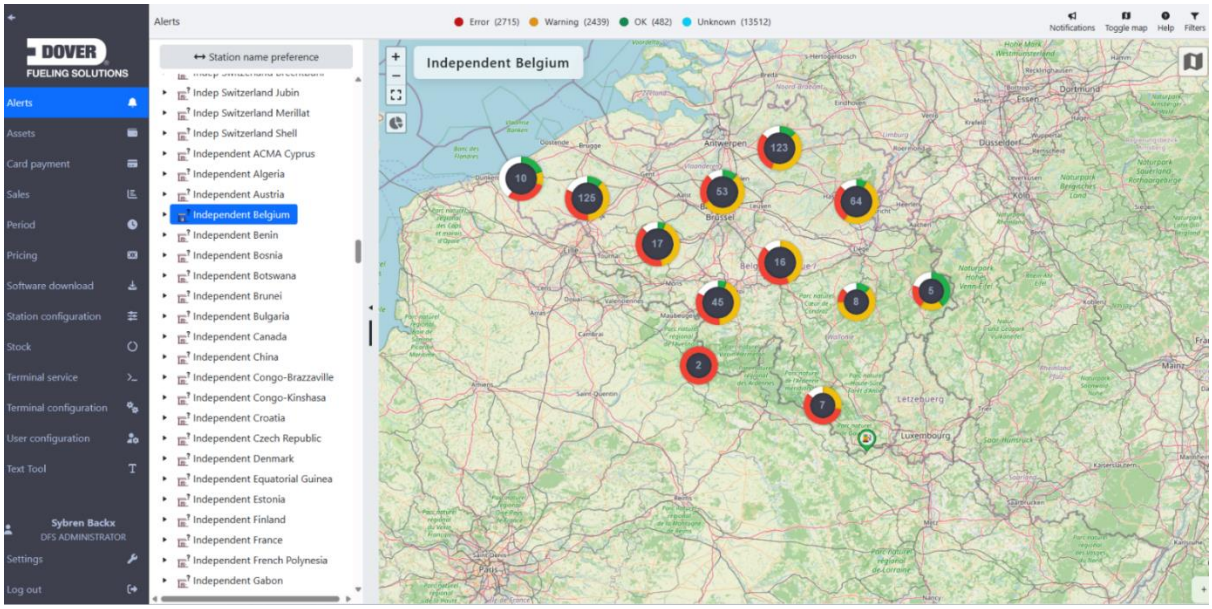
## 4.4. Conclusion

With the backend infrastructure, database schemas, and stored procedures now updated, the project has transitioned from data preparation to active frontend integration. The system can now store, retrieve, and manually override geographic coordinates, ensuring the map component has a reliable data source.

# 5. Leaflet map functionalities

An interactive Leaflet dashboard map is to be added to the existing alerts module in One Portal and should provide a client with the option to display all the stations of a CouCom (Country Company) on the map. Leaflet is an open-source library that can be used to build interactive maps inside ONE Portal. To build this map, the address data of a station will be used to translate it into coordinates to position the station in the right position. The alerts module of ONE Portal will be given an alternative view besides the tab view, where clients can visually get a high-level overview of all their stations of a network, together with their status. This section will go over all the functionalities of the map.

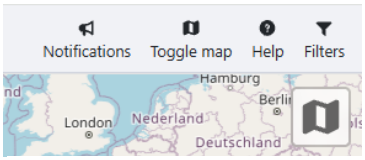
The Leaflet map inside OP will be an extension to the alerts module. It can be found as an extra component inside the body component of the alerts module. The following image is an example of what the map looks like when selecting the CouCom of Independent Belgium.




*15 Leaflet alerts map*

## 5.1. Toggling between tab and map view

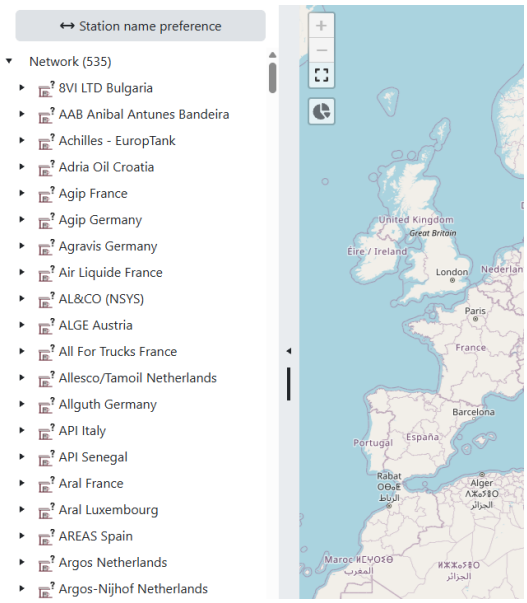
To make the map visible, you first need to press the “Toggle map” button in the top right of the screen. Pressing this button, flips the view from the original tab view to the map view and vice versa.



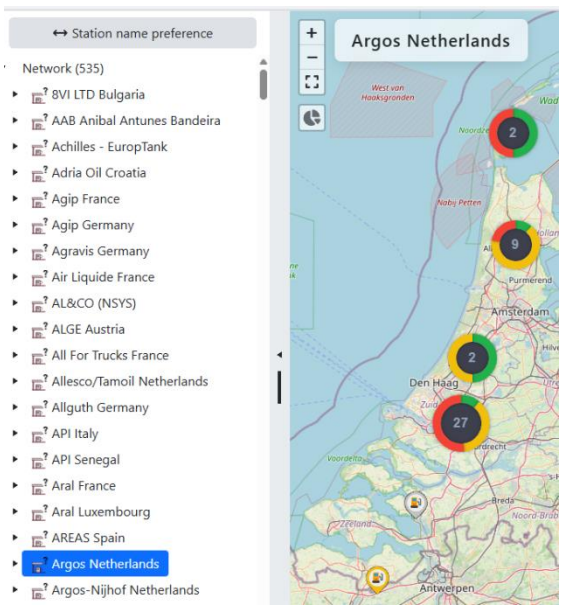

*16 Map toggle button*

## 5.2. Displaying a CouCom on the map

The map is tied to the already existing tree on the left that clients use to navigate. Displaying something on the map happens on Coucom level. To display a CouCom, simply click the CouCom you want to show inside the tree. In the example below, in the first image, you see the default view when nothing is selected, then in the second picture, you see the map after CouCom “Argos Netherlands” has been selected in the tree.




*17 Default map view nothing selected*

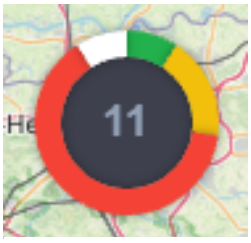



*18 Map view with Coucom selected*

## 5.3. Interpreting the map

### Clusters

After selecting a CouCom in the tree, you will be met with 2 different icons. The first one is a pie chart, which represents a cluster of stations. A cluster can be interpreted as a group of stations here. In the middle of the pie chart, you will see a number representing how many stations there are inside the cluster.



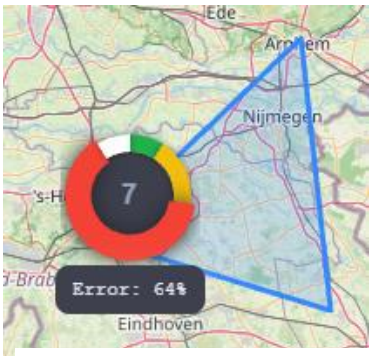

*19 Example cluster icon*

Then there are the colored segments of the pie chart which represent the status of the stations inside the cluster. You will see:

- green for status “OK”,
- yellow for status ‘Warning”,
- red for status “Error”
- white for status “Undefined”.

Also notice that when you hover the cluster, you get a blue polygon drawing on the map which represents the border or region the stations reside in.

When you hover over one of the segments, like in the second image. The number in the middle shows how many stations there are in that colored segment. So in this case, there are 7 stations with a status of “Error”. At the bottom, you get a percentage of how many stations belong to the segment. In this case, 64% of the stations have the status of “Error”.




*20 Example hovered cluster icon*

### Marker Icons

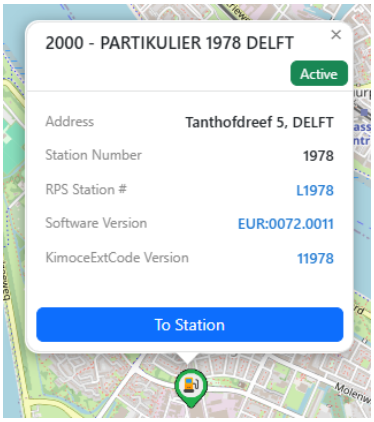
You will also see the following icon in 4 different colors

-  : A station with the status “OK”.
-  : A station with the status “Warning”.
-  : A station with the status “Error”.
-  : A station with the status “Undefined”.

## 5.4. Marker popups

### Popup Card

The station markers on the map are clickable. When clicking such a marker, then a popup will appear above that marker. Here you will find some basic station information if available.




*21 Popup example*

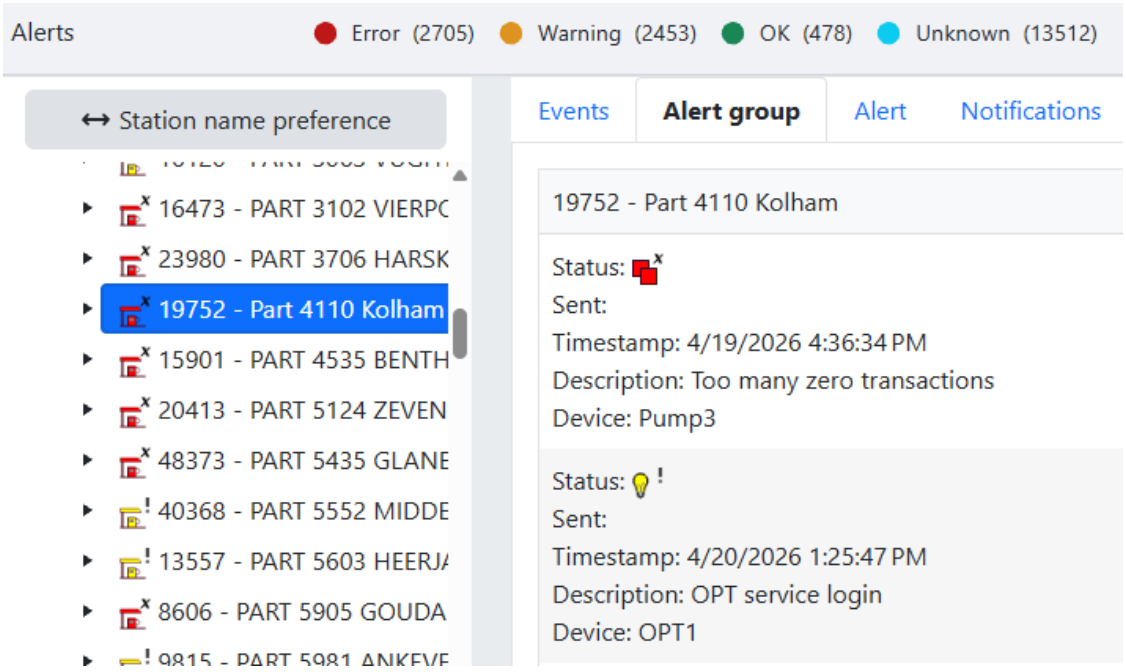
Popup card content:

- Station Name
- Address
- Station RPS number
- KimoceExtCode
- Station state badge
- Station Number
- Software version

The green station badge on the top right of a popup represents the state of the station. The badge colors can be green, yellow, red or white based on the state of the station.

### Drill Down Button

On the bottom of the popup card is a blue button that says, “To Station”. When this button is clicked, the tree on the left is programmatically searched for the right station. When the station belonging to clicked marker has been found, then the view switches automatically to tab view so you can see the station details of that station. This way a user can easily drill down on any station to figure out what is going on.




*22 Screen after pressing drill down button*

## 5.5. Marker Clustering

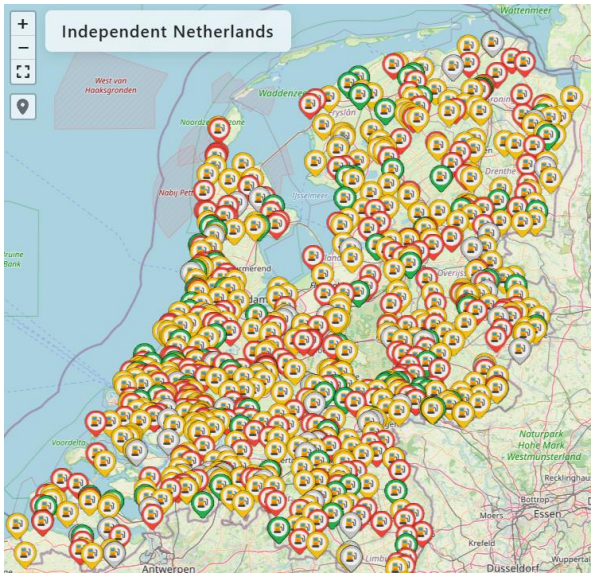
In the following 2 images, you can see the CouCom of “Independent Netherlands” in 2 different views available in the map. As you can see, there is a big difference between the 2 in terms of visibility. This CouCom contains about 900 stations that all need to be displayed on the map, and this can get cluttered really fast, making it impossible to click the right station.

To solve this and keep visibility high, marker clustering was implemented in the map. This means that stations that are close together get grouped together based on how close they are on the screen.

In the second image you can see clustering enabled and you get the stations nicely grouped together inside a pie chart. This solves the problem from before, where selecting the correct station in the clutter is impossible. Now you can drill down on a station by clicking a cluster/pie chart. By doing this, the map will zoom in and recalculate the clusters. The clusters will continue to shrink with each click, until you are able to see the marker of the station you want.

### Toggle Button

Enabling the marker clustering functionality is easily done by pressing the last button in the top left corner of the map. You can see the icon of the clustering button flipping between a pie chart and a regular marker. This way, a user can still set the map to the view he prefers. Because in some cases, the CouCom will not contain 900 stations but maybe 40. In that case, clustering might not be needed and disabling it might be preferable.




*23 Map without marker clustering enabled*


*24 Map with marker clustering enabled*

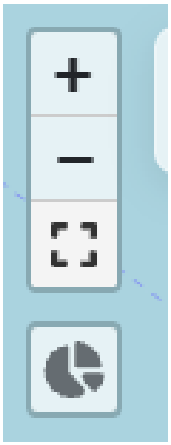
**Performance**
In addition to more visibility, marker clustering also improves the performance of the map significantly. Having to display 900 stations with their popup component reference concurrently is a heavy load for OP. Marker clustering solves this issue partly.

Other steps have been implemented to make using the map a smooth experience. One of these steps is lazy loading the popup components. Instead of creating and remembering all popups of all markers, it is much more performant to only create the popup when a marker is clicked. This way a popup is only created when it is needed. Also, when a popup is deselected, it gets unbound and destroyed to clean up the map.

The last measure to improve performance on the map is to make use of viewport-based loading. Markers that are off screen serve no purpose and drag down performance. So consequently, only markers and clusters on screen are loaded in.

## 5.6. Fullscreen

The map also provides a functionality to put the map in full screen. This functionality is nice when the map is put on a large monitor in a room. This ensures that the whole CouCom can be displayed on the map at once. This option can be found in the top left corner of the map, as the third option.



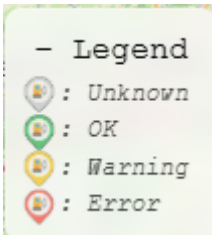
*25 Map controls*

## 5.7. Zoom functions

The map can be scrolled with the mouse wheel. But alternatively, this can also be done by clicking the “+” and “-” buttons in the top left corner.

## 5.8. Map Legend

Another smaller feature is the map legend in the bottom right corner of the map. Here you can see what the color code means of the status of a station marker. By default, this legend is hidden under a small “+” icon in the bottom right corner of the map.

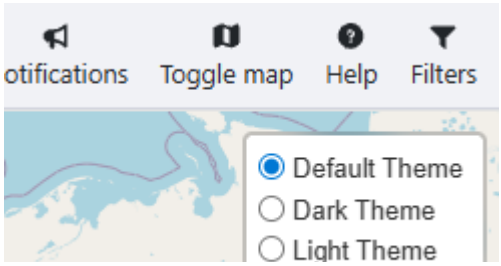



*26 Map legend*

## 5.9. Tile Providers

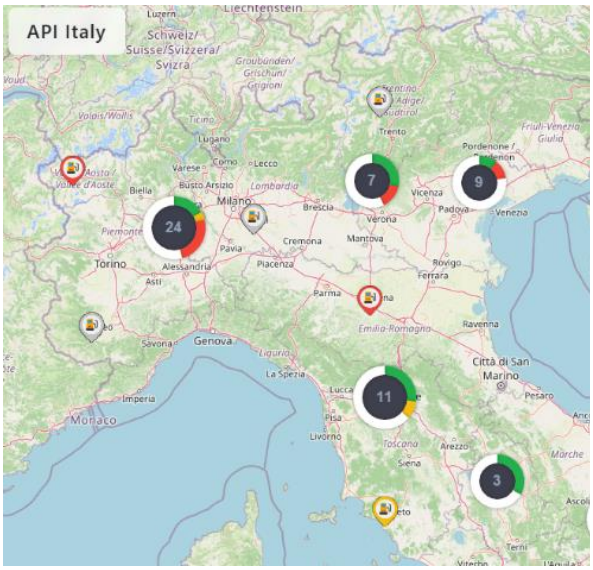
A tile provider is an online server or service that serves pre-generated, fixed-size image or vector map tiles. This will determine the style or what kind of map you are looking at.

For the map in OP, there is a selection of 3 different tile providers. These tile providers are all open-source options and can be used free of cost. They can be activated in the top right corner of the map by hovering over the button and selecting the preferred tile provider.

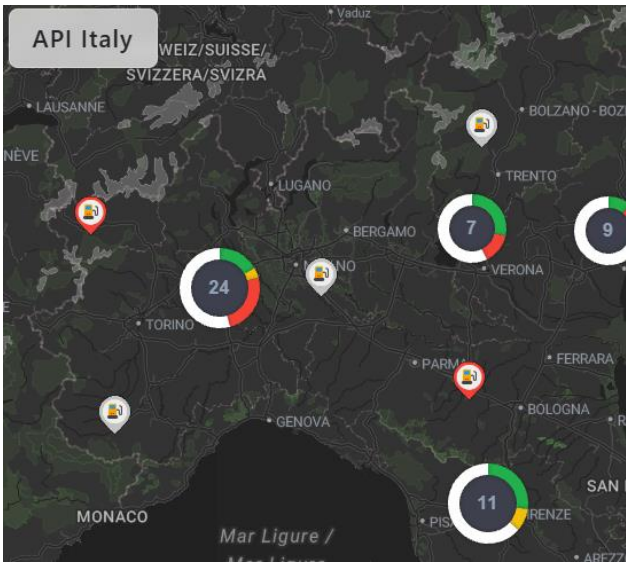



*27 Tile provider controls*

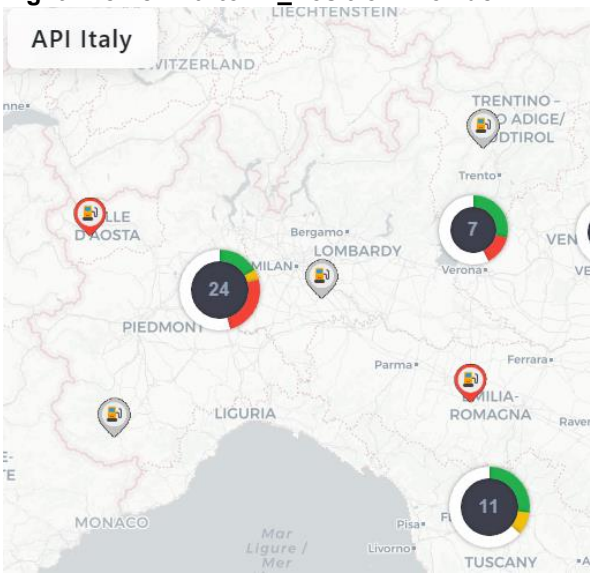
**Default Theme / OpenStreetMap Default Provider**




**Dark Theme / Stadia_AlidadeSmoothDark Provider**

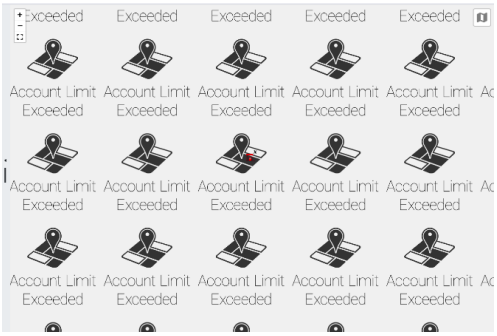



**Light Theme / CartoDB_Positron Provider**



**Tile provider policies**

The following section is straight from the Nominatim OpenStreetMap policy page. It describes the requirements that need to be met to make use of their tile service and public API. Not following these guidelines might result in access being blocked and receiving the following screen.




*28 Account suspension screen*

**Requirements**

- No heavy uses (an absolute **maximum of 1 request per second**).
- Provide a valid **HTTP Referer** or **User-Agent** identifying the application (stock User-Agents as set by http libraries will not do).
- Clearly display attribution as suitable for your medium.
- Data is provided under the ODbL license which requires to share alike (although small extractions are likely to be covered by fair usage / fair dealing).

**Websites and apps**

Use that is directly triggered by the end-user (for example, user searches for something) is ok, provided that your number of users is moderate. Note that the usage limits above apply **per website/application**: the sum of traffic by all your users should not exceed the limits.

Apps must make sure that they can switch the service at our request at any time (in particular, switching should be possible without requiring a software update). If at all possible, **set up a proxy** and also enable caching of requests.

**Note:** periodic requests from apps are considered bulk geocoding and as such are strongly discouraged. It may be okay if your app has very few users and applies appropriate caching of results. Make sure you stay well below the API usage limits.

**Bulk Geocoding**

As a general rule, bulk geocoding of larger amounts of data is **not** encouraged. If you have regular geocoding tasks, please, look into alternatives below. Smaller one-time bulk tasks may be permissible, if these additional rules are followed

- limit your requests to **a single thread**
- limited to 1 machine only, **no distributed scripts** (including multiple Amazon EC2 instances or similar)
- Results **must be cached** on your side. Clients sending repeatedly the same query may be classified as faulty and blocked.

**Unacceptable use**

The following uses are strictly forbidden and will get you banned:

- **Auto-complete search** This is not yet supported by Nominatim and you must not implement such a service on the client side using the API.
- **Systematic queries** This includes reverse queries in a grid, searching for complete lists of postcodes, towns etc. and downloading all POIs in an area. If you need complete sets of data, get it from the OSM planet or an extract.
- **Scraping of details** The details page is there for debugging only and may not be downloaded automatically.

## 5.10. Zoom limits and map boundaries

To limit tile usage and prevent our access being revoked, zoom limits and map boundaries are put into place. These measures make sure that a user loads as little tiles as possible.

### MAP BOUNDARIES

When a CouCom is selected in the tree, the map will automatically fly to the boundaries of that CouCom. Originally, a median of all coordinates of all stations in a CouCom were used to pan the map to the right position. But this approach broke the animations when selecting a station marker. When you select a station marker, the map pans so the station is nicely in the middle of the screen. But with the median set and the boundaries fixed to that view, the map could not pan. As a solution, the corners of the boundaries are calculated, and extra padding is added to not hinder any animations. With this approach, all the stations nicely fit the screen, animations are not obstructed and the boundaries prevent users from any needless scrolling. Thus, preventing any unnecessary map tiles to be loaded.

```
// Find corners of the boundary
coordinates.forEach(coord => {
    const lat = coord[0] !== 0 ? Number(coord[0]) : NaN;
    const lng = coord[1] !== 0 ? Number(coord[1]) : NaN;

    if (!isNaN(lat) && !isNaN(lng)) {
        latMin = Math.min(latMin, lat);
        latMax = Math.max(latMax, lat);
        lngMin = Math.min(lngMin, lng);
        lngMax = Math.max(lngMax, lng);
    }
});

/** Set the corners*/
let cornerBottomLeft: L.LatLngExpression = [latMin, lngMin];
let cornerTopRight: L.LatLngExpression = [latMax, lngMax];

/** The bounds without padding are needed to nicely fit all stations */
const boundsForZoom = L.latLngBounds(cornerBottomLeft, cornerTopRight);
```

*29 Map boundaries code*

### ZOOM LIMITS

The zoom level of the map is locked to a default value when nothing is selected. It is not possible to move the map in this case This prevents any unnecessary zooming and loading of map tiles. When a CouCom is selected and the map finishes panning to its location. Then zoom limits are set, only allowing a user to zoom in and out by a configured amount.

# 6. Conclusion

The primary objective of this internship was to transform the existing tree-view alerts module into a geographic dashboard. This process began with a feasibility study to get access to the coordinates of the stations. By creating a custom Python-based geocoding tool using the Nominatim API, over 28,000 stations were successfully processed. After that, the backend infrastructure including the SQLite databases and PM APIs was extended to support the new station coordinates data.

### PROJECT EVALUATION

Looking back at the goals set in the project plan, the internship ended up delivering a functional, clear map integration within ONE Portal.

### FUTURE PROOF:

Through automated geocoding in the Station Configurator, the PM database supports the creation of new stations with their geographic coordinates.

### UX & PERFORMANCE:

By implementing marker clustering, lazy loading, the dashboard remains performant even when displaying the maximum amount of 900 stations in OP.

### POLICY COMPLIANCE:

The integration respects external API policies through solutions like, zoom limits and map boundaries.

# 7. References


Agafonkin, V. (2010–2025). *leafletjs*. Retrieved from leafletjs: https://leafletjs.com/

Agafonkin, V. (n.d.). *Layer Groups and Layers Control*. Retrieved from leafletjs: https://leafletjs.com/examples/layers-control/

Census, U. S. (n.d.). *Census*. Retrieved from Census: https://geocoding.geo.census.gov/geocoder/

community, N. d. (n.d.). *Nominatim 5.2.0 Manual*. Retrieved from Nominatim 5.2.0 Manual: https://nominatim.org/release-docs/latest/admin/Installation/#hardware

D.Holemans. (2020, September 28). Feasibility Study One Portal Database.

Geoapify. (2020, April 02 ). *Map libraries comparison: Leaflet vs MapLibre GL vs OpenLayers - trends and statistics*. Retrieved from Geoapify: https://www.geoapify.com/map-libraries-comparison-leaflet-vs-maplibre-gl-vs-openlayers-trends-and-statistics/#:~:text=Leaflet%20is%20ideal%20for%20simple,and%20enterprise%2Dlevel%20mapping%20projects

Geoapify. (2026). *Geoapify*. Retrieved from Geoapify: https://www.geoapify.com/tools/geocoding-online/

HERE. (2026, january 27). *HERE Geocoding and Search - Developer Guide*. Retrieved from HERE Geocoding and Search - Developer Guide: https://www.here.com/docs/bundle/geocoding-and-search-api-developer-guide/page/topics/quick-start.html

maplibre-gl-js. (n.d.). *maplibre-gl-js examples*. Retrieved from maplibre-gl-js: https://maplibre.org/maplibre-gl-js/docs/examples/

Matrix, B. (2025, December 4). *How Does Tokheim S.A.S. Company Work?* Retrieved from BCG Matrix: https://matrixbcg.com/blogs/how-it-works/tokheim

nominatim. (n.d.). *nominatim search*. Retrieved from nominatim: https://nominatim.openstreetmap.org/ui/search.html

nominatim. (n.d.). *Open-source geocoding with OpenStreetMap data*. Retrieved from nominatim: https://nominatim.org/

ODonohue, D. (2025 , August 17). *Complete QGIS Beginners Roadmap*. Retrieved from mapscaping: https://mapscaping.com/complete-qgis-beginners-roadmap/

Openlayers. (n.d.). *Openlayers*. Retrieved from Openlayers: https://openlayers.org/

osmfoundation. (n.d.). *Tile Usage Policy*. Retrieved from operations osmfoundation: https://operations.osmfoundation.org/policies/tiles/

Seelmann, S. (n.d.). *leaflet-extras*. Retrieved from leaflet-extras: https://leaflet-extras.github.io/leaflet-providers/preview/